

MathImpl — Tech Stack & Deployment Specification

1. Tech Stack

Frontend

Concern	Choice
Framework	React 18
Build tool	Vite
Language	TypeScript (strict)
Styling	Tailwind CSS v4
Routing	React Router v6
Global state	Zustand
Forms	React Hook Form
Charts	Recharts
Icons	Lucide React
Node runtime	Node 20 (via nvm)

Backend

Concern	Choice
Language	Go 1.22+
HTTP framework	chi (https://github.com/go-chi/chi) v5
Auth	JWT (golang-jwt/jwt v5)
PostgreSQL driver	pgx (https://github.com/jackc/pgx) v5
Query builder	sqlc (https://sqlc.dev/) + database/sql
MongoDB driver	mongo-go-driver (https://www.mongodb.com/docs/drivers/go/) v1.15
Migration tool	golang-migrate (https://github.com/golang-migrate/migrate)
Config	environment variables via <code>os.Getenv</code> / <code>.env</code> (joho/godotenv)
Logging	<code>log/slog</code> (stdlib, structured JSON)
API style	REST (JSON)

Database

Concern	Choice
Relational DB	PostgreSQL 16 — structured data (users, roles, sessions, analytics)
Document DB	MongoDB 7 — flexible/nested data (assessments, lesson configs, logs)
Connection pooling	pgBouncer (https://www.pgouncer.org/) (PostgreSQL)

Infrastructure

Concern	Choice
Container runtime	Docker + Docker Compose
Reverse proxy	Nginx
TLS	Let's Encrypt (Certbot)
CI/CD	GitHub Actions
Cloud option	AWS (see §4)

2. Repository Layout

```

mathimpl/
├── frontend/           # React + Vite app
├── backend/            # Go API server
│   ├── cmd/server/    # main.go entry point
│   ├── internal/      # domain packages (auth, users, analytics ...)
│   ├── db/
│   │   ├── postgres/  # sqlc queries + migrations
│   │   └── mongo/     # MongoDB collections + seed helpers
│   └── Dockerfile
├── docker-compose.yml  # local dev (all services)
└── TECH_STACK.md

```

3. Server Specifications — Self-Hosted (Bare Metal / VPS)

3.1 Staging

Pre-production QA, internal team only.

App Server (Frontend + Backend)

Spec	Value
CPU	2 vCPU
RAM	2 GB
Storage	40 GB SSD
OS	Ubuntu 22.04 LTS
Open ports	22, 80, 443
Services	Nginx, Go API (:8080)
Domain	staging.mathimpl.app

Database Server (PostgreSQL + MongoDB)

Spec	Value
CPU	2 vCPU
RAM	2 GB
Storage	40 GB SSD
OS	Ubuntu 22.04 LTS
Open ports	5432 & 27017 (internal only)
Services	PostgreSQL 16, pgBouncer (:6432), MongoDB 7
Backup	Daily dump → compressed archive, 3-day retention

3.2 Production

Live traffic — redundant app nodes, DB replica set, pgBouncer pooling.

App Servers (×2, load balanced)

Spec	Value
CPU	4 vCPU
RAM	8 GB
Storage	80 GB SSD
OS	Ubuntu 22.04 LTS
Open ports	22, 80, 443
Services	Nginx, Go API (:8080)
Domain	app.mathimpl.app

Database Server — Primary

Spec	Value
CPU	4 vCPU
RAM	16 GB
Storage	200 GB SSD
OS	Ubuntu 22.04 LTS
Open ports	5432 & 27017 (internal only)
Services	PostgreSQL 16 (primary), pgBouncer, MongoDB 7 (replica set primary)
Backup	Daily dump, 7-day retention + WAL archiving (PostgreSQL)

Database Server — Secondary / Replica

Spec	Value
CPU	2 vCPU
RAM	8 GB
Storage	200 GB SSD
OS	Ubuntu 22.04 LTS
Open ports	5432 & 27017 (internal only)
Services	PostgreSQL 16 (streaming replica), MongoDB 7 (replica set secondary)
Usage	Read-heavy analytics queries routed here

Network

- All servers on the same private network; DB ports never exposed to the internet
- App servers have public IPs; DB servers have private IPs only
- Firewall: app → DB primary (5432, 27017), DB primary ↔ DB secondary (replication)
- TLS at Nginx (Let's Encrypt); Go ↔ Nginx on loopback HTTP

4. Server Specifications — AWS Option

Drop-in AWS equivalents. Same application config; only the infrastructure layer changes.

4.1 Staging (AWS)

Component	AWS Service	Spec / Notes
Frontend (static)	AWS Amplify	Connected to staging branch; auto-deploy on push; custom domain <code>staging.mathimpl.app</code>
Backend API	ECS Cluster (Fargate)	0.5 vCPU / 1 GB, 1 task, no autoscaling
Secrets & env vars	ECS Task Definition	Environment variables and sensitive values injected directly into task definition
PostgreSQL	Amazon RDS PostgreSQL 16	<code>db.t3.micro</code> , 20 GB gp3, single-AZ, 3-day backups
MongoDB	MongoDB Atlas M10 / Amazon DocumentDB	2 vCPU / 2 GB, no replica
pgBouncer	ECS sidecar container	Transaction-mode pooling, port 6432
Object storage	Amazon S3	Uploads, avatars, exported reports; bucket <code>mathimpl-objects-staging</code>
Load balancer	Application Load Balancer (ALB)	HTTPS 443 → HTTP 8080
TLS	AWS Certificate Manager (ACM)	Attached to ALB and Amplify domain
DNS	Route 53	A-record → ALB; CNAME → Amplify

4.2 Production (AWS)

Component	AWS Service	Spec / Notes
Frontend (static)	AWS Amplify	Connected to <code>main</code> branch; auto-deploy on push; custom domain <code>app.mathimpl.app</code> ; WAF enabled
Backend API	ECS Cluster (Fargate)	2 vCPU / 4 GB, min 2 tasks, autoscale to 6 at CPU 60%
Secrets & env vars	ECS Task Definition	Environment variables and sensitive values injected directly into task definition
PostgreSQL	Amazon RDS PostgreSQL 16	<code>db.r6g.large</code> , 200 GB gp3, Multi-AZ, read replica, 7-day backups + PITR
MongoDB	MongoDB Atlas M30 / Amazon DocumentDB	4 vCPU / 16 GB, 3-node replica set; analytics on secondary
pgBouncer	ECS sidecar container	Transaction-mode pooling, port 6432
Object storage	Amazon S3	Uploads, avatars, exported reports; bucket <code>mathimpl-objects-prod</code> ; versioning enabled
Load balancer	Application Load Balancer (ALB)	HTTPS only, HTTP → HTTPS redirect
TLS	AWS Certificate Manager (ACM)	Attached to ALB and Amplify domain
DNS	Route 53	A-record → ALB; CNAME → Amplify
Container registry	Amazon ECR	Stores versioned Go API images

5. Server Summary

Self-Hosted

Environment	Role	CPU	RAM	Storage
Staging	App server	2 vCPU	2 GB	40 GB SSD
Staging	DB server (PG + Mongo)	2 vCPU	2 GB	40 GB SSD
Production	App server ×2	4 vCPU	8 GB	80 GB SSD
Production	DB primary (PG + Mongo)	4 vCPU	16 GB	200 GB SSD
Production	DB secondary (PG + Mongo)	2 vCPU	8 GB	200 GB SSD

AWS

Environment	Role	AWS Service	Spec
Staging	Frontend	AWS Amplify	staging branch, auto-deploy
Staging	API	ECS Fargate	0.5 vCPU / 1 GB
Staging	PostgreSQL	RDS PostgreSQL 16	db.t3.micro / 20 GB
Staging	MongoDB	Atlas M10 / DocumentDB	2 vCPU / 2 GB
Production	Frontend	AWS Amplify	main branch, auto-deploy + WAF
Production	API	ECS Fargate	2 vCPU / 4 GB (×2–6)
Production	PostgreSQL	RDS PostgreSQL 16	db.r6g.large / 200 GB, Multi-AZ
Production	MongoDB	Atlas M30 / DocumentDB	4 vCPU / 16 GB, 3-node RS
Staging	Object storage	Amazon S3	mathimpl-objects-staging
Production	Object storage	Amazon S3	mathimpl-objects-prod , versioning on